408计算机组成原理大题解题模板

专题一：Cache映射、地址划分、命中率与平均访问时间

必会专题

适用综合应用题及选择题

本专题的目标是：**看到Cache计算题，能够迅速识别题型，并按照固定步骤完成计算。**

408的Cache综合题通常不是单纯考一个公式，而是将Cache结构、地址划分、程序访存和性能计算结合起来。

例如：

- 2020年：组相联映射、Tag与LRU位、数组访问Cache的缺失次数。
- 2023年：二维数组地址、Cache组号、空间局部性。
- 2025年：Cache地址划分、数组访问缺失率、平均访存时间。

因此，下面按照实际做题顺序整理。

一、Cache综合题的万能起手式

1. 题目识别

遇到以下关键词：

- Cache数据区容量
- 主存块大小
- 直接映射、全相联、n路组相联
- 主存地址位数
- Cache组号、标记位Tag、块内偏移
- Cache命中率、缺失率

第一反应：

先算Cache行数，再算组数，最后划分地址。

2. 必背公式

假设：

- 主存地址长度：m位
- Cache数据区容量：C字节
- 主存块大小：B字节
- 组相联路数：E

第一步：计算Cache行数

$$\boxed{L=\frac{C}{B}}$$

其中：

- L：Cache总行数
- C：Cache数据区容量
- B：每个Cache行的数据区大小，也就是主存块大小

第二步：计算Cache组数

$$\boxed{S=\frac{L}{E}=\frac{C}{B\times E}}$$

其中：

- 直接映射：$E=1$，因此每组1行

- n路组相联：每组有n行
- 全相联：只有1组

第三步：计算块内地址位数

$$\boxed{b = \log_2 B}$$

注意：按字节编址时，B必须用字节表示。

第四步：计算组号位数

$$\boxed{s = \log_2 S}$$

第五步：计算Tag位数

$$\boxed{t = m - s - b}$$

这五个公式可以直接背下来，绝大多数Cache地址划分题都是从这里开始的。

二、三种Cache映射方式的地址划分

1. 直接映射

每个主存块只能映射到Cache中唯一的一行。

$$\boxed{\text{Cache行号} = \text{主存块号} \bmod L}$$

地址划分为：

直接映射地址结构



2. 组相联映射

主存块只能映射到指定的一组，但可以存放在该组的任意一行。

$$\boxed{\text{Cache组号} = \text{主存块号} \bmod S}$$

地址划分为：

组相联映射地址结构



3. 全相联映射

每个主存块可以放入Cache的任意一行。

因此：

- 没有行号或组索引字段
- 仅需要Tag和块内地址

$$\boxed{t = m - b}$$

4. 三种映射方式比较

映射方式	组数	地址结构
直接映射	L	Tag＋行号＋块内地址
E路组相联	L/E	Tag＋组号＋块内地址
全相联	1	Tag＋块内地址

记忆口诀：直接找一行，组相联找一组，全相联随便放。

三、题型一：求Cache组数、Tag位数、块内地址

这是最基础的计算题，应当做到不需要思考就能列式。

【典型例题】

某计算机主存地址为32位，按字节编址，Cache数据区容量为32KB，主存块大小为64B，采用8路组相联映射。

求：

1. Cache共有多少行?
2. Cache共有多少组?
3. 块内地址占多少位?
4. Cache组号占多少位?
5. Tag占多少位?

第一步：计算Cache行数

$$L = \frac{32 \times 1024}{64} = 512$$

第二步：计算Cache组数

$$S = \frac{512}{8} = 64$$

第三步：计算块内地址位数

$$b = \log_2 64 = 6$$

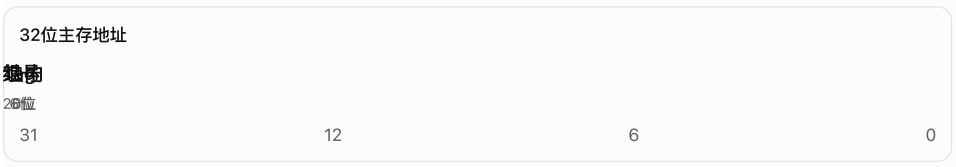
第四步：计算组号位数

$$s = \log_2 64 = 6$$

第五步：计算Tag位数

$$t = 32 - 6 - 6 = 20$$

最终得到：



考试固定步骤：行数 → 组数 → 块内位数 → 组号位数 → Tag位数。

四、题型二：已知主存地址，求Cache组号

这部分非常重要。

有的题目并不会明确要求划分整个地址，而是直接问：

> 主存地址0x12345678映射到Cache的哪一组?

此时不需要把32位地址全部转换为二进制。

1. 万能公式

$$\boxed{\text{主存块号} = \left\lfloor \frac{\text{主存地址}}{B} \right\rfloor}$$

$$\boxed{\text{Cache组号} = \text{主存块号} \bmod S}$$

合并：

$$\boxed{\text{Cache组号} = \left\lfloor \frac{\text{主存地址}}{B} \right\rfloor \bmod S}$$

其中：

- B：主存块大小
- S：Cache组数

2. 典型例题

已知：

- 主存地址为0x00000140
- 主存块大小64B
- Cache共有64组

求该地址对应的Cache组号。

首先把十六进制转换为十进制：

$$0x140 = 320$$

求主存块号：

$$\text{主存块号} = \left\lfloor \frac{320}{64} \right\rfloor = 5$$

求Cache组号：

$$\text{组号} = 5 \bmod 64$$

所以：

$$\boxed{\text{Cache组号} = 5}$$

3. 十六进制快速计算法

如果块大小、组数都是2的整数次幂，可以直接取地址中的对应位。

例如：

- 主存块64B：低6位是块内偏移
- Cache有64组：接着的6位是组号

因此：

$$\boxed{\text{组号} = (A \gg 6)\ \&\ 0x3F}$$

其中：

- `>>6`：右移6位
- `&0x3F`：保留最低6位

对于熟悉C语言的你，这种写法应该更容易记忆。

但考试书写时，建议同时写出数学表达式，过程更清楚。

五、题型三：求Cache命中率、缺失率、缺失次数

这类题是综合题的重要部分，尤其喜欢结合C语言数组考查。

1. 必背公式

$$\boxed{\text{命中率} = \frac{\text{命中次数}}{\text{总访问次数}}}$$

$$\boxed{\text{缺失率} = \frac{\text{缺失次数}}{\text{总访问次数}}}$$

二者满足：

$$\boxed{\text{命中率} + \text{缺失率} = 1}$$

真正容易出错的地方不是公式，而是：

怎样确定Cache缺失次数?

2. 连续访问数组的固定模板

如果数组连续存储、Cache初始为空、访问期间没有导致重复调入的冲突，并且每个首次访问的主存块都会装入Cache，那么：

$$\boxed{\text{缺失次数} = \text{访问的不同主存块数量}}$$

注意这里有前提，特别是涉及写操作时，需要考虑是否按写分配。

情况一：数组首地址恰好对齐主存块边界

假设：

- 数组总长度4096B
- 主存块64B

$$N = \frac{4096}{64} = 64$$

因此，顺序读取数组时产生64次Cache缺失。

情况二：数组首地址没有对齐

假设：

- 数组总长度4096B
- 主存块64B
- 数组首地址距离所在主存块起始位置32B

那么：

$$N = \left\lceil \frac{32 + 4096}{64} \right\rceil$$

$$\boxed{N = 65}$$

因为数组从某个主存块的中间开始，最后还会多占用一个主存块。

这就是经常出现的“多一次缺失”。

3. 更通用的公式

设：

- 数组首地址：A
- 主存块大小：B
- 连续访问的数据总量：D

首先计算起始偏移：

$$o = A \bmod B$$

然后：

$$\boxed{N_{\text{块}} = \left\lceil \frac{o + D}{B} \right\rceil}$$

这是特别值得背诵的一个公式。

4. 数组访问次数怎么计算?

看C语言代码。

第一种：只读数组

```c
C
for (int i = 0; i < 1024; i++) {
    sum += a[i];
}
```

每个元素读取一次。

因此，单就数组数据访问而言：

$$\text{Cache访问次数} = 1024$$

第二种：既读又写

```c
C
for (int i = 0; i < 1024; i++) {
    a[i] = 2 * a[i];
}
```

一般需要：

1. 读取 `a[i]`
2. 计算 `2*a[i]`
3. 写回 `a[i]`

因此在题目按一次读、一次写统计，并不考虑特殊编译优化时：

$$\boxed{\text{Cache访问次数} = 1024 \times 2 = 2048}$$

假设Cache初始为空，按写分配且不会再次因冲突换出已加载数据，数组覆盖64个主存块。

那么：

- 首次读取每块时缺失一次
- 同一块后续读写均命中

所以总缺失次数为64。

$$\text{缺失率} = \frac{64}{2048}$$

$$\boxed{\text{缺失率} = 3.125\%}$$

这里必须注意：如果题目指定统计“读缺失率”，分母就不能把写访问也算进去。

六、题型四：平均Cache访问时间 AMAT

这类题可以说是最适合直接套公式的。

1. 必背公式

$$\boxed{AMAT = T_{\text{hit}} + MR \times MP}$$

其中：

- $AMAT$：平均访存时间
- T_{hit}：Cache命中时间
- MR：缺失率
- MP：额外缺失损失（Miss Penalty）

2. 典型例题

假设：

- Cache命中时间为2个时钟周期
- 缺失损失为200个时钟周期
- 缺失率为5%

那么：

$$AMAT = 2 + 0.05 \times 200$$

$$\boxed{AMAT = 12\text{个时钟周期}}$$

3. 一定要注意题目对“缺失时间”的定义

如果题目给的是：

> Cache命中时间2个周期，Cache未命中时总访问时间202个周期。

则：

$$AMAT = (1 - MR) \times T_{\text{hit}} + MR \times T_{\text{miss}}$$

例如缺失率5%：

$$AMAT = 0.95 \times 2 + 0.05 \times 202$$

$$\boxed{AMAT = 12}$$

结果相同，但计算依据不同。

记忆：给“缺失损失”，用加法式；给“未命中总耗时”，用加权平均式。

七、题型五：Cache替换算法与替换次数

常见题目会给出：

- Cache初始为空
- 采用直接映射或组相联
- 采用FIFO或LRU
- 给出一串主存块访问序列
- 要求计算命中次数、缺失次数、替换次数

1. 第一步：确定每个主存块所属组

若Cache有S组：

$$\boxed{\text{组号} = \text{主存块号} \bmod S}$$

先把访问序列按组分类。

不同组之间不会互相替换。

2. 第二步：只模拟发生冲突的组

例如某Cache采用2路组相联，组数为4。

主存块访问序列：

$$0,\ 4,\ 8,\ 0,\ 12$$

这些块的组号都是：

$$0 \bmod 4 = 0$$

$$4 \bmod 4 = 0$$

$$8 \bmod 4 = 0$$

$$12 \bmod 4 = 0$$

因此，所有访问都集中在第0组。

3. LRU模拟

LRU：替换最长时间没有被访问的块。

2路组相联，访问第0组

访问	组内内容（旧→新）	结果
0	0	缺失，装入
4	0，4	缺失，装入
8	4，8	缺失，替换0
0	8，0	缺失，替换4
12	0，12	缺失，替换8

因此：

- Cache缺失次数：5
- Cache命中次数：0
- 替换次数：3

为什么替换只有3次?

因为前两次装入空行，不需要替换已有数据。

$$\boxed{\text{替换次数} \neq \text{缺失次数}}$$

这是一个重要区别。

FIFO与LRU的核心区别

- FIFO：谁最早进入，替换谁。命中不改变进入次序。
- LRU：谁最长时间没被使用，替换谁。命中会更新使用次序。

八、题型六：Cache写策略与容量计算

1. 写策略必须区分两组概念

策略	含义
直写 Write Through	写Cache时同时更新主存
回写 Write Back	先写Cache，替换脏块时再写主存
写分配 Write Allocate	写缺失时将对应块装入Cache
非写分配 No Write Allocate	写缺失时不因此装入Cache，通常直接写下一级

注意：直写和回写讨论**如何更新主存**；写分配和非写分配讨论**写缺失时是否装入Cache块**。这是两组不同的策略。

2. Cache总容量计算

如果题目要求的是包含Tag、有效位等元数据的总容量，就不能只用数据区容量作答。

设：

- Cache共有L行
- 每行存储B字节数据
- Tag占t位
- 有效位1位
- 脏位d位
- 每行还需要r位替换状态（按题目给定的具体实现）

则：

$$\boxed{C_{\text{total}} = L(8B + t + 1 + d + r)}$$

单位是位。

如果替换状态按整组保存，就应先计算每组替换状态，再乘组数，而不能机械地按每行计算。

3. 常见附加位

附加位	作用
有效位 Valid	判断Cache行的数据是否有效
修改位 Dirty	回写策略下标记数据是否被修改
替换状态位	为LRU等替换策略保存必要状态
Tag	判断该行存储的是哪个主存块

九、考前必背：Cache题型速查表

问什么	立即想到的公式或步骤
Cache共有多少行	$L = C/B$
Cache共有多少组	$S = C/(BE)$
块内地址占几位	$b = \log_2 B$
组号占几位	$s = \log_2 S$
Tag占几位	$t = m - b - s$
某地址映射到哪一组	$\lfloor A/B \rfloor \bmod S$
连续数组覆盖多少块	$\lceil ((A \bmod B) + D)/B \rceil$
Cache缺失率	缺失次数 / 访问总次数
Cache命中率	$1 -$ 缺失率
平均访存时间	命中时间＋缺失率×额外缺失损失
LRU替换谁	最长时间未被访问的块
FIFO替换谁	最早进入的块
回写是否需要脏位	需要记录是否修改
直写是否需要脏位	通常不需要记录待回写状态

十、给你一组考场模拟题

下面是一道按照408真题风格编写的综合练习，建议你先不看提示，独立列式。

Cache综合应用题 建议12分钟

某计算机按字节编址，主存物理地址为32位，数据Cache采用4路组相联映射，数据区容量为16KB，主存块大小为64B，初始时Cache为空。

已知一维数组 `a[256]`，每个元素占4B，首地址为 `0x00001020`。程序依次读取数组中的全部元素，无其他数据访问干扰，也不发生Cache块的重复换入。

（1）Cache共有多少组?

（2）Cache组号、块内地址、Tag分别占几位?

（3）a[10]的主存地址是多少?

（4）a[10]对应的Cache组号是多少?

（5）整个数组读取过程中发生多少次Cache缺失?

（6）整个数组读取的Cache缺失率是多少?

最后总结：本专题真正需要练到什么程度?

就408考试而言，我认为你至少应该达到以下水平：

- 看到Cache容量、块大小、路数，10～20秒内能够列出组数、Tag等计算式。
- 看到十六进制地址，能迅速确定块内偏移和组号。
- 看到一段数组循环，能分清元素数、字节数、覆盖块数、访问次数。
- 看到命中时间和缺失损失，能立即写出AMAT公式。
- 看到LRU/FIFO，知道应该先算映射组，再只模拟发生冲突的组。

我特别想提醒你一点：**Cache大题的难点经常不在计算，而在搞清楚题目到底问的是行数、组数、主存块号，还是组号。**

你之前就遇到过把Cache行数和行号位数混淆的问题。这类错误完全可以靠固定模板纠正：先写清楚每个量的物理意义，再代入计算。

下一专题是**虚拟存储器、TLB与Cache综合计算**。它与本专题联系非常紧密，尤其是2023、2025年那种把虚拟地址、数组与Cache放在一起考查的题目。

408计算机组成原理大题解题模板

专题二：虚拟存储器、TLB、Cache综合计算

必会专题

408综合应用题高价值题型

本专题重点解决以下问题：

1. 虚拟地址和物理地址怎么划分?
2. 如何利用页表、TLB完成地址转换?
3. TLB采用组相联映射时，怎么计算组号和Tag?
4. 怎样判断TLB命中、Cache命中、缺页异常?
5. 为什么有时可以直接用虚拟地址计算Cache组号?
6. 虚拟存储器、Cache和C语言数组结合时，应该怎么解题?

这类题在408中很有代表性。例如：

- 2018年：TLB、Cache、虚实地址转换综合题。
- 2021年：TLB组相联映射与LRU替换。
- 2023年：分页、二维数组、Cache组号与缺页次数。
- 2025年：虚拟地址与Cache索引、数组访问和缺失率。

核心思路：先分页，后Cache；先划分地址，再代入具体数值。

一、虚拟存储综合题的万能起手式

1. 题型识别

题目出现：

- 虚拟地址VA、物理地址PA
- 页大小、页内偏移
- 页号、页框号
- 页表、页表项PTE
- TLB、快表
- Cache组号、标记位
- 缺页异常

第一反应：**先确定页大小，再划分虚拟地址和物理地址。**

2. 必背公式

假设：

- 虚拟地址长度为m位
- 物理地址长度为n位
- 页面大小为$P = 2^p$字节

第一步：求页内偏移位数

$$\boxed{p = \log_2 P}$$

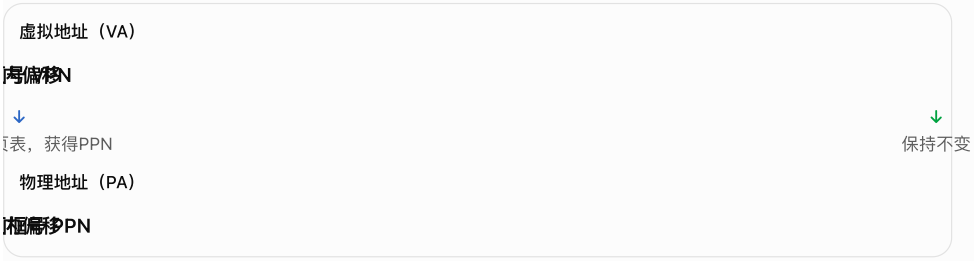
第二步：求虚页号位数

$$\boxed{\text{VPN位数} = m - p}$$

第三步：求物理页框号位数

$$\boxed{\text{PPN位数} = n - p}$$

因此，地址结构如下。



第四步：写出地址转换公式

$$\boxed{\text{VPN} = \left\lfloor \frac{VA}{P} \right\rfloor}$$

$$\boxed{\text{页内偏移} = VA \bmod P}$$

通过页表查到物理页框号PPN后：

$$\boxed{PA = PPN \times P + \text{页内偏移}}$$

这三个式子就是分页地址转换最核心的模板。

二、题型一：虚拟地址和物理地址的划分

【典型例题】

某计算机：

- 虚拟地址长度32位
- 物理地址长度28位
- 页面大小4KB
- 按字节编址

求：

1. 页内偏移占多少位?
2. 虚页号占多少位?
3. 物理页框号占多少位?

4. 虚拟页数和物理页框数分别是多少?

第一步：页内偏移位数

$$4KB = 4 \times 1024B = 2^{12}B$$

因此：

$$\boxed{p = 12}$$

第二步：虚页号位数

$$32 - 12 = 20$$

因此：

$$\boxed{\text{VPN占20位}}$$

第三步：物理页框号位数

$$28 - 12 = 16$$

因此：

$$\boxed{\text{PPN占16位}}$$

第四步：虚拟页数

$$N_{\text{虚页}} = 2^{20}$$

第五步：物理页框数

$$N_{\text{页框}} = 2^{16}$$

最终地址划分：

地址	高位	低位
虚拟地址（32位）	VPN：20位	页内偏移：12位
物理地址（28位）	PPN：16位	页内偏移：12位

必背结论

虚拟地址和物理地址的页内偏移位数相同，但VPN和PPN的位数不一定相同。

因为页内偏移决定的是页面内部的位置，而地址转换只改变页面所在的位置，不改变数据在页面内部的偏移。

三、题型二：根据虚拟地址求物理地址

这是最基本的地址转换题。

1. 固定解题模板

给定虚拟地址VA：

第一步：

$$VPN = \left\lfloor \frac{VA}{P} \right\rfloor$$

第二步：

$$Offset = VA \bmod P$$

第三步：查页表，找到VPN对应的PPN。

第四步：

$$\boxed{PA = PPN \times P + Offset}$$

2. 【典型例题】

已知：

- 虚拟地址32位
- 页面大小4KB
- 虚拟地址为 `0x12345ABC`
- 页表中该虚页对应的物理页框号为 `0x0A7BC`
- 对应页表项有效，且页面已在主存

求物理地址。

第一步：确定页内偏移

$$4KB = 2^{12}B$$

因此，页内偏移占12位。

12位刚好对应3位十六进制数。

所以：

$$VA = \underbrace{12345}_{VPN}\ \underbrace{ABC}_{Offset}H$$

得到：

$$VPN = 0x12345$$

$$Offset = 0xABC$$

第二步：查页表

已知：

$$PPN = 0x0A7BC$$

第三步：拼接物理地址

因为页内偏移保持不变：

$$PA = 0x0A7BC000 + 0xABC$$

因此：

$$\boxed{PA = 0x0A7BCABC}$$

考试快捷技巧

当页面为4KB时：

- 低3位十六进制数是页内偏移
- 剩下的高位是虚页号
- 查到物理页框号后，替换高位
- 低3位十六进制数原样保留

例如：

`VA = 12345ABCH`

查到：

`PPN = 0A7BCH`

则：

`PA = 0A7BCABCH`

只需要完成高位替换，运算非常快。

但这个技巧只适用于页面大小为2^{4k}字节等能够整齐对应十六进制数位的情况。

四、题型三：TLB组相联映射，求组号和Tag

这类题可以直接套Cache的思路，但有一个非常重要的区别：

TLB通常是根据虚页号VPN进行映射，而不是直接根据整个虚拟地址进行映射。

1. 必背公式

假设：

- 虚拟地址为m位
- 页面大小为2^p字节
- TLB采用E路组相联
- TLB一共有N个表项

第一步：计算虚页号位数

$$\boxed{v = m - p}$$

第二步：计算TLB组数

$$\boxed{S = \frac{N}{E}}$$

第三步：计算TLB组号位数

$$\boxed{s = \log_2 S}$$

第四步：计算TLB Tag位数

$$\boxed{t = v - s}$$

所以：

$$\boxed{t = m - p - s}$$

这里默认TLB使用VPN低位作为组索引。

2. TLB与Cache地址划分的区别

项目	TLB	Cache
映射对象	虚页号VPN	主存块号
组号计算	VPN mod TLB组数	主存块号 mod Cache组数
Tag来源	VPN高位	参与Cache查找的地址高位
存储内容	地址转换信息	主存块数据

TLB地址结构

虚拟地址划分：TLB采用组相联映射



3. 【典型例题】

某计算机：

- 虚拟地址32位
- 页面大小4KB
- TLB共有32个表项
- TLB采用4路组相联

求TLB组号和Tag位数。

第一步：求虚页号位数

$$v = 32 - 12 = 20$$

第二步：求TLB组数

$$S = \frac{32}{4} = 8$$

第三步：求TLB组号位数

$$s = \log_2 8 = 3$$

第四步：求TLB Tag位数

$$t = 20 - 3 = 17$$

因此：

$$\boxed{\text{TLB Tag为17位，组号为3位}}$$

完整虚拟地址划分为：

$$\boxed{17 + 3 + 12 = 32}$$

五、题型四：TLB采用LRU，判断替换哪个页表项

这一类非常适合用固定步骤做。

1. 题目识别

题目出现：

- TLB采用2路组相联
- 采用LRU替换算法
- TLB初始为空
- 给出虚页号访问序列
- 问哪个表项会被替换

第一反应：**先计算所有VPN对应的TLB组号，再单独模拟发生冲突的组。**

2. 固定步骤

第一步：计算TLB组数。

$$S = \frac{\text{TLB表项数}}{\text{路数}}$$

第二步：计算每个VPN对应的组号。

$$\boxed{\text{组号} = VPN \bmod S}$$

第三步：把映射到同一组的VPN提取出来。

第四步：根据LRU更新使用顺序。

第五步：当某组已经装满时，替换最久未被使用的表项。

3. 【2021年408真题考法】

TLB采用2路组相联，共8组，使用LRU替换算法。

访问的虚页号依次为：

$$10,\ 12,\ 16,\ 7,\ 26,\ 4,\ 12,\ 20$$

求被替换的虚页号。

第一步：计算TLB组号

$$\text{组号} = VPN \bmod 8$$

得到：

VPN	10	12	16	7	26	4	12	20
组号	2	4	0	7	2	4	4	4

第二步：找到发生替换的组

第4组依次访问：

$$12,\ 4,\ 12,\ 20$$

由于是2路组相联，一组最多存放2个表项。

第三步：模拟LRU

访问	第4组内容（旧→新）	结果
12	12	缺失，装入
4	12、4	缺失，装入
12	4、12	命中，更新LRU
20	12、20	缺失，替换4

因此：

$$\boxed{\text{被替换的虚页号为}4}$$

这里最容易出错的地方

注意第三次访问12时：

$$12,\ 4,\ \boxed{12}$$

12被再次访问，变成最近使用的表项。

因此，最后访问20时应该替换4，而不是12。

LRU命中也要更新使用顺序，FIFO命中不更新进入次序。

六、题型五：TLB未命中、Cache未命中、缺页异常的判断

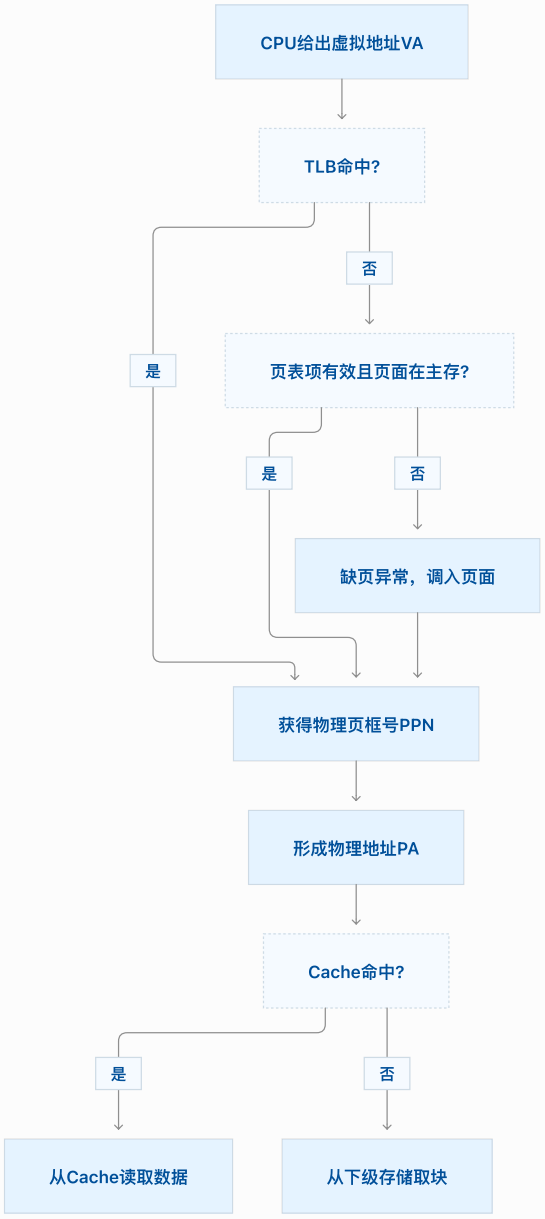
这是一个非常容易混淆的概念题。

你需要把三个层次分开。

1. 三种“未命中”含义不同

情况	含义	是否必然缺页
TLB未命中	快表中没有对应地址转换信息	否
Cache未命中	Cache中没有需要的主存块	否
缺页异常	目标页面尚未装入主存	是

2. 地址转换过程



图中是逻辑处理流程。实际机器可能让TLB查找与部分Cache索引并行进行，具体以题目给定的数据通路为准。

3. 必背判断

情况一：TLB命中，Cache命中

直接获得物理地址，并从Cache中获取数据。

情况二：TLB命中，Cache未命中

地址转换成功，但目标主存块不在Cache中。

需要从下级存储取得数据块。

情况三：TLB未命中，但页表项有效、页面在内存

需要查页表获得地址转换结果。

此时不会发生缺页异常。

情况四：TLB未命中，页表显示页面不在主存

发生缺页异常。

操作系统将页面调入主存，更新页表等信息，再按规定恢复执行。

一个重要推论

在通常的正确一致性机制下：

如果真正发生缺页，原先主存中就没有目标页面，不应认为CPU可以直接从当前普通数据Cache中得到该页所需的数据。

但“TLB未命中”本身完全不能推出“Cache一定未命中”。

七、题型六：计算访问主存的次数

这类题经常出现在TLB与页表相关的选择题和综合题中。

1. 最基础的模型

假设：

- 一级页表
- 不考虑Cache
- 一次主存访问读取一个页表项或数据
- 不发生缺页异常

TLB命中

不需要到主存中查页表。

只需要访问主存取得数据。

$$\boxed{\text{主存访问次数} = 1}$$

TLB未命中

需要：

1. 访问主存中的页表，查找页表项。
2. 访问主存中的实际数据。

因此：

$$\boxed{\text{主存访问次数} = 2}$$

2. 多级页表的情况

假设采用二级页表，且不考虑Cache。

TLB未命中时：

1. 访问一级页表。
2. 访问二级页表。
3. 访问实际数据。

所以：

$$\boxed{\text{主存访问次数} = 3}$$

推广到k级页表：

$$\boxed{\text{TLB未命中访问次数} = k + 1}$$

前提是不发生缺页、每一级页表访问都需要一次主存访问。

3. 如果同时考虑Cache呢?

必须看题目给出的条件。

例如：

- TLB命中，数据Cache命中：可能不需要访问主存。
- TLB命中，数据Cache未命中：需要从主存或更低层次取数据。
- TLB未命中，页表项可能也在Cache中：查页表不一定真的访问主存。

因此，考试中只要题目出现“访问主存次数”，一定先看它有没有写：

不考虑Cache。

或者：

所有页表项均在主存中，且访问页表需要一次主存访问。

千万不要在同时考虑TLB和Cache时，直接死套1次、2次、3次。

八、题型七：虚拟地址中的哪些位可以作为Cache组号?

这是本专题最重要、也最值得专门背诵的综合题型之一。

你可能会发现：

有些题目给的是虚拟地址，却直接要求计算Cache组号。

你会疑惑：

Cache不是使用物理地址吗？为什么还没完成地址转换，就能确定Cache组号？

原因在于：

分页地址转换不会改变页内偏移。

1. 必背判断条件

假设：

- 页面大小为2^p字节
- Cache块大小为2^b字节
- Cache共有2^s组

那么：

- 页内偏移占p位
- Cache块内偏移占b位
- Cache组索引占s位

如果：

$$\boxed{p \geq b + s}$$

那么Cache组索引所需要的地址位全部落在页内偏移中。

这些位在VA和PA中完全相同。

因此，可以直接用虚拟地址的相应低位确定物理索引Cache的组号。

换一个更容易记忆的形式：

$$p \geq b + s$$

等价于：

$$2^p \geq 2^b \times 2^s$$

所以：

$$\boxed{\text{页面大小} \geq \text{主存块大小} \times \text{Cache组数}}$$

又因为：

$$\text{Cache组数} = \frac{\text{数据区容量}}{\text{块大小} \times \text{路数}}$$

所以还能化成：

$$\boxed{\text{页面大小} \geq \frac{\text{Cache数据区容量}}{\text{路数}}}$$

这三个形式其实是同一个条件。

2. 【典型例题】

假设：

- 虚拟地址32位
- 页面大小4KB
- Cache数据区容量32KB
- 8路组相联
- 主存块大小64B

问：虚拟地址中哪些位可以直接作为Cache组号？

第一步：页内偏移位数

$$p = \log_2 4096 = 12$$

第二步：Cache块内偏移位数

$$b = \log_2 64 = 6$$

第三步：Cache组数

$$S = \frac{32KB}{64B \times 8} = 64$$

组号位数：

$$s = \log_2 64 = 6$$

第四步：判断是否满足条件

$$b + s = 6 + 6 = 12$$

因为：

$$p = 12$$

所以：

$$\boxed{p = b + s}$$

全部组索引位都落在页内偏移中。

因此：

$$\boxed{VA[11:6]\text{可以直接作为Cache组号}}$$

3. 示意图

虚拟地址（32位）



绿色与黄色共12位，恰好是4KB页面的页内偏移，因此转换前后保持不变。

4. 如果不满足条件怎么办?

例如：

- 页面大小4KB：$p = 12$
- Cache块大小64B：$b = 6$
- Cache共有256组：$s = 8$

那么：

$$b + s = 14 > 12$$

说明Cache组索引需要使用物理地址中超出页内偏移范围的位。

此时不能仅凭VA与PA的低12位相同，就确定完整的物理Cache组号。

必须结合具体地址转换结果或题目规定的Cache索引方式。

必背结论

$$\boxed{p \geq b + s \Rightarrow VA\text{中相应位可直接用于物理}Cache\text{组索引}}$$

注意：这只能说明组索引位在转换前后相同，**不意味着Cache的Tag也能直接使用VPN**。物理标记Cache仍需正确的物理Tag。

九、题型八：二维数组＋分页＋Cache综合大题

这是非常有代表性的综合考法。

我们参照2023年408第43题的主要条件来分析。

【综合例题】

某计算机：

- 虚拟地址32位
- 页面大小4KB
- Cache采用4路组相联
- Cache数据区8KB
- 主存块32B

定义数组：

```c
int a[24][64];

for (int i = 0; i < 24; i++) {
    for (int j = 0; j < 64; j++) {
        a[i][j] = 10;
    }
}
```

已知：

- `int` 占4B
- 数组采用行优先存储
- 数组首地址为 `0x00422000`
- 执行前数组的页面均不在主存
- 执行期间不发生页面置换

求：

1. 数组占用多少个页面?
2. 发生多少次缺页异常?
3. `a[1][0]` 的虚拟地址是多少?
4. `a[1][0]` 所在主存块对应Cache哪一组?
5. 数组访问具有什么局部性?

第一问：数组占用多少页面?

第一步：计算数组元素个数

$$24 \times 64 = 1536$$

第二步：计算数组总字节数

$$1536 \times 4 = 6144B$$

第三步：计算起始页内偏移

页面大小4KB。

首地址为：

$$0x00422000$$

它的低12位为0，说明数组首地址恰好位于页面起始位置。

第四步：计算占用页面数

$$N = \left\lceil \frac{6144}{4096} \right\rceil$$

因此：

$$\boxed{N = 2\text{页}}$$

第二问：发生多少次缺页异常?

题目明确说明：

- 数组页面执行前均不在主存
- 执行期间不发生页面置换

每个页面第一次被访问时发生一次缺页。

因为数组跨越两个页面，所以：

$$\boxed{\text{缺页次数} = 2}$$

缺页地址分别是两个页面内第一次实际被访问的地址：

$$\boxed{0x00422000, \quad 0x00423000}$$

注意：**缺页异常次数与Cache缺失次数不是一回事。**

一个页面包含很多主存块：

$$\frac{4096}{32} = 128$$

所以访问一个页面可能出现许多次Cache缺失，但不意味着发生许多次缺页。

第三问：`a[1][0]`的虚拟地址

二维数组行优先地址公式：

$$\boxed{A(a[i][j]) = A_0 + (i \times C + j) \times w}$$

其中：

- $C = 64$：列数
- $w = 4B$：元素大小

代入：

$$A(a[1][0]) = 0x00422000 + (1 \times 64 + 0) \times 4$$

$$= 0x00422000 + 256$$

256的十六进制为：

$$256 = 0x100$$

因此：

$$\boxed{A(a[1][0]) = 0x00422100}$$

第四问：对应Cache哪一组?

第一步：计算Cache组数

$$S = \frac{8KB}{32B \times 4}$$

$$S = \frac{8192}{128} = 64$$

第二步：计算块内地址位数

$$b = \log_2 32 = 5$$

第三步：计算组号位数

$$s = \log_2 64 = 6$$

因此，Cache组号需要地址第5～10位。

第四步：判断能否使用虚拟地址

页面大小4KB：

$$p = 12$$

而：

$$b + s = 5 + 6 = 11$$

满足：

$$11 \leq 12$$

因此，这些组索引位位于页内偏移范围，可以直接用VA求组号。

第五步：计算组号

`a[1][0]` 的虚拟地址为：

$$0x00422100$$

其页内偏移为：

$$0x100 = 256$$

计算：

$$\text{组号} = \left\lfloor \frac{256}{32} \right\rfloor \bmod 64$$

$$= 8 \bmod 64$$

所以：

$$\boxed{\text{Cache组号} = 8}$$

第五问：具有什么局部性?

数组采用行优先存储，并且：

```c
C
for (int i = 0; i < 24; i++) {
    for (int j = 0; j < 64; j++) {
        a[i][j] = 10;
    }
}
```

内层循环每次增加 `j`，访问连续数组元素。

因此具有良好的空间局部性。

但每个数组元素仅被访问并赋值一次，若只讨论这些数组元素，并没有明显的时间局部性。

所以可以答：

> 数组访问具有空间局部性，因为连续访问相邻存储单元；不具有明显的时间局部性，因为每个数组元素仅被访问一次。

这类局部性判断题，一定先看题目讨论的是数组元素还是整个程序（循环变量、指令等也可能被反复访问）。

十、题型九：页表大小、页表项数量、二级页表

虽然这种题不一定每年作为计组大题单独考查，但经常与虚拟存储器综合出现。

1. 一级页表大小

假设：

- 虚拟地址m位
- 页面大小2^p字节
- 每个页表项E字节

虚拟页数：

$$\boxed{N = 2^{m-p}}$$

一级页表存放所有虚页的表项时：

$$\boxed{C_{\text{页表}} = 2^{m-p} \times E}$$

2. 典型例题

- 32位虚拟地址
- 页面大小4KB
- 页表项4B

求一级页表大小。

虚页号位数：

$$32 - 12 = 20$$

虚拟页数：

$$2^{20}$$

页表大小：

$$C = 2^{20} \times 4B$$

$$= 2^{22}B$$

因此：

$$\boxed{C = 4MB}$$

3. 二级页表怎么分?

仍然设：

- 32位虚拟地址
- 页面大小4KB
- 页表项4B
- 各级页表都占一个4KB页面

第一步：求每张页表有多少表项

$$\frac{4096}{4} = 1024 = 2^{10}$$

因此，页表索引占10位。

第二步：求虚页号总位数

$$32 - 12 = 20$$

第三步：划分二级页表

$$20 = 10 + 10$$

得到：

32位虚拟地址，二级页表



必背结论

二级页表的虚拟地址通常分为：

$$\boxed{\text{一级索引} + \text{二级索引} + \text{页内偏移}}$$

如果题目给出每张页表大小和页表项大小，优先算：

$$\boxed{\text{每张页表项数} = \frac{\text{页表大小}}{\text{页表项大小}}}$$

然后取$\log_2$求索引位数。

十一、本专题易错点总结

容易混淆的概念	正确理解
VPN与PPN	VPN是虚页号，PPN是物理页框号
VPN与TLB组号	VPN用于查TLB，组号通常由VPN低位计算
TLB组号与Cache组号	是两套独立的映射编号
TLB未命中与缺页	TLB未命中不代表页面不在主存
Cache未命中与缺页	Cache未命中不代表缺页
页大小与块大小	页面通常远大于Cache块，但以题目为准
VA与PA的低位	页内偏移始终不变
VA与PA的高位	一般需要页表转换，不能直接视为相同
LRU与FIFO	LRU命中会更新使用次序，FIFO不会
缺页次数与Cache缺失次数	缺页按页面统计，Cache缺失按块访问统计
页表项大小与页面大小	页表项存映射信息，页面存放数据或页表内容

十二、考前必背公式速查

这部分建议单独摘出来反复背。

题目问法	固定公式
页内偏移位数	$p = \log_2 P$
虚页号位数	$m - p$
物理页框号位数	$n - p$
虚页号	$\lfloor VA/P \rfloor$
页内偏移	$VA \bmod P$
物理地址	$PPN \times P + Offset$
TLB组数	N/E
TLB组号	$VPN \bmod S$
TLB Tag位数	$m - p - \log_2 S$
Cache组数	$C/(B \times E)$
Cache组号	$\lfloor PA/B \rfloor \bmod S$
VA能否直接提供Cache组号	$p \geq b + s$
连续数据覆盖页数	$\lceil ((A \bmod P) + D)/P \rceil$
一级页表大小	$2^{m-p} \times E_{\text{PTE}}$
每张页表的表项数	页表大小 / 页表项大小
单级页表且TLB未命中访问主存次数	2次（不考虑Cache及缺页）
k级页表且TLB未命中访问主存次数	$k+1$次（相同简化条件下）

其中：

- P：页面大小（字节）
- m：虚拟地址位数
- n：物理地址位数
- B：主存块大小
- C：Cache数据区容量
- E：Cache或TLB相联度（按所在公式）
- S：Cache或TLB组数（按所在公式）
- D：连续数据总字节数

十三、最后做一道综合模拟题

分页＋TLB＋Cache综合练习 建议10～15分钟

某计算机按字节编址，虚拟地址为32位，物理地址为28位，页面大小4KB。

TLB采用4路组相联，共有32个表项；数据Cache采用8路组相联，数据区容量为32KB，主存块大小64B。

现CPU访问虚拟地址 `0x12345ABC`，页表中对应的物理页框号为 `0x0A7BC`，且该页面已在主存中。

（1）虚页号和页内偏移分别是多少？

（2）对应的物理地址是多少？

（3）TLB有多少组？组号和Tag各占几位？

（4）该虚页映射到TLB的第几组？

（5）Cache共有多少组？块内偏移、组号、Tag各占几位？

（6）该数据对应的Cache组号是多少？

（7）能否直接利用VA的部分位确定Cache组号？

本专题的最终背诵重点

你现在时间比较紧，我建议优先把以下四个模板背熟：

第一类：地址转换

$$VPN = \lfloor VA/P \rfloor, \qquad Offset = VA \bmod P$$

$$PA = PPN \times P + Offset$$

第二类：TLB映射

$$\text{组号} = VPN \bmod S$$

第三类：Cache索引能否直接使用VA

$$\boxed{\text{页内偏移位数} \geq \text{Cache组号位数} + \text{块内偏移位数}}$$

第四类：分页＋数组综合

$$\boxed{\text{数组总字节数} = \text{元素个数} \times \text{每元素字节数}}$$

$$\boxed{\text{覆盖页数} = \left\lceil \frac{\text{首页内偏移} + \text{数组总字节数}}{\text{页面大小}} \right\rceil}$$

我认为本专题最需要你建立的条件反射是：

看见VA，先拆VPN与Offset；看见TLB，拿VPN算组号；看见Cache，拿主存块号算组号；看见两者结合，先判断页内偏移是否覆盖Cache索引位。

这样遇到跨章节综合题，就不容易在第一步陷入混乱。

下一专题是**专题三：C语言数组、机器级代码与存储访问综合题**。这类题会把数组地址计算、指令长度、寻址方式、大小端、Cache访问放到一起，非常值得专项训练。